



INDIAN INSTITUTE OF INFORMATION TECHNOLOGY DESIGN AND MANUFACTURING KURNOOL

Department of Computer Science and Engineering

MID-SEMESTER PROJECT REPORT

Composing Context Compression for Long-Horizon LLM Agents

*Controlled Composition and Critical-Fact Retention
on top of ACON*

A report submitted in partial fulfilment of the requirements
for the award of the degree of

Bachelor of Technology

Computer Science and Engineering

Submitted by

123CS0006 Ayush Rahate

123CS0054 Rohit Chauhan

Project Supervisor

Dr. N. SRINIVAS NAIK

Department of Computer Science and Engineering
Indian Institute of Information Technology Design and Manufacturing Kurnool
Kurnool, Andhra Pradesh – 518008

September 2026

1. Introduction

LLM agents solve long-horizon tasks by alternating actions and observations over tens of steps, and every tool output and intermediate thought is appended to the prompt. The context therefore grows without bound. This causes two problems: inference memory grows with the KV cache, and long, partly stale contexts distract the model and degrade its decisions [1]. Losing a single file path or API token can derail an entire trajectory, so compression must be aggressive and precise at the same time.

Existing methods compress one kind of context at a time: the interaction history, the latest observation, or a document. Agents need several of these stages at once, and it is not known how they interact. This project takes ACON [1] as the base method. We reproduce it and SAC [2] on our own hardware, and study how compression stages should be *composed* and *evaluated*.

2. Literature Review

Generative, prompt-level compression. ACON [1] uses an LLM to rewrite the history once it exceeds a threshold T_{hist} and to shorten observations longer than T_{obs} . The compression guideline is optimised in natural language: trajectories that succeed without compression but fail with it are contrasted to produce feedback, and the compressor is then distilled into smaller models. On AppWorld [7] with gpt-4.1, history compression reaches 56.5% accuracy against 56.0% with no compression, and peak tokens drop by 26–54% across three benchmarks.

Soft, KV-level compression. SAC [2] selects anchor tokens from the context, marks them with a learned embedding, and uses bidirectional attention so that their key-value pairs absorb the whole context, without autoencoding pretraining. On MRQA with Llama-3.2-1B at $15\times$ compression, it reaches 54.95 in-domain F1, above EPL (51.52).

Extractive, code-specific pruning. SWE-Pruner [3] places a 0.6B line-level classifier between a coding agent and its tools. The agent writes a goal hint that conditions which lines of retrieved code are kept. On SWE-Bench Verified it cuts tokens by 23–38% while raising the resolve rate by 1.2–1.4 points.

Token pruning and RL-trained memory. LLMingua [4] removes low-information tokens using a small language model. It is task-agnostic, and in ACON’s AppWorld comparison it reaches 39.3% accuracy for history compression. MEM1 [5] and MemAgent [6] instead train the agent with RL to maintain a bounded memory. MemAgent extrapolates from an 8K window to 3.5M-token QA with under 10% loss, but these methods require updating the agent’s weights.

Property	ACON	SAC	SWE-Pruner	LLMLingua	MEM1/MemAgent	Proposed
Output form	Text	KV cache	Text lines	Text tokens	Text memory	Text
Agent weights frozen	Yes	Yes	Yes	Yes	No	Yes
History compression	Yes	No	No	Yes	Merged	Yes
Observation compression	Yes	No	Yes	Yes	Merged	Yes
Composition studied	Partly	No	No	No	No	Yes
Intrinsic quality metric	No	No	No	No	No	Yes (CFR)

Table 1: Comparison of context-compression families. “Partly”: ACON reports a combined setting in an appendix ablation only. “Merged”: one memory holds both.

3. Implementation and Results

Setup. All runs use one NVIDIA RTX 2000 Ada (16 GB). For ACON, models are served locally through Ollama: the agent is `qwen2.5:14b` and the compressor `qwen2.5:7b`. We use the AppWorld `train_history_tiny` split (38 tasks) with seed 42 and temperature 0. For SAC, we use the authors’ Llama-3.2-1B checkpoints on MRQA.

ACON reproduction (preliminary). Table 2 reports our AppWorld runs. The fast runs use 8 tasks with at most 12 iterations. The full history-compression run had completed 18 of 38 tasks at the time of writing, so it is compared with the baseline on the same 18 tasks. On these, history compression solved 9 tasks against 7 and used 24.5% fewer agent input tokens. The change is not uniform: it solved 5 tasks the baseline failed and failed 3 the baseline solved. On the 8-task fast runs, both compression settings finished slightly sooner than the baseline, so the latency overhead ACON reports (Section 4, gap (a)) did not appear at this scale.

These numbers carry three caveats. First, Ollama served both models with a 4096-token context, so long baseline prompts were truncated and part of any gain may come from staying under this limit. Second, thresholds are 512/256 rather than the paper’s 4096/1024, because the paper values rarely fire on this split. Third, each setting was run with a single seed.

SAC reproduction. Evaluating the released checkpoints reproduces the published macro-averaged MRQA scores to within 0.2 F1: at $15\times$ we obtain 54.98 in-domain / 39.25 out-of-domain F1 (published 54.95 / 39.26), and at $5\times$ 63.70 / 47.71 (published 63.63 / 47.72).

Run	Tasks	Max iter.	Solved	Avg iter.	Agent input tok.	Wall clock (s)
Baseline, no compression	8	12	1	11.00	269,935	953.9
History compression ($T_{\text{hist}} = 512$)	8	12	2	10.75	232,749	882.8
Observation compression ($T_{\text{obs}} = 256$)	8	12	1	11.00	261,422	873.6
Baseline, full split	38	30	12 (31.6%)	23.7	2,972,376	7312
Baseline, matched subset	18	30	7	22.22	1,283,906	not logged
History compression, matched subset	18	30	9	19.50	969,389	not logged

Table 2: Preliminary ACON runs on AppWorld (qwen2.5:14b agent, qwen2.5:7b compressor, seed 42). Wall-clock time is not logged for the matched subset because the baseline’s progress log is incomplete.

SAC×ACON pilot (preliminary). We transferred two ACON ideas into SAC’s latent space. *Failure-gated distillation* (FGD) adds a KL term from the full-context decoder only on examples where it beats the compressed student. *Task-guided anchors* (TGA) append the question to each encoder chunk so anchors can attend to it; only anchor KV is kept, so the compression ratio is unchanged. All arms start from the released 15× pretrain adapter, train for 5,000 steps with the same seed and data order, and differ only in the loss or input (Table 3).

Run	ID F1	OOD F1	Δ ID vs. R0 (95% CI)	Δ OOD vs. R0 (95% CI)
R0: SAC fine-tuning (matched control)	51.37	38.62	control	control
R2: + FGD (gated KL, $\lambda = 1$)	51.15	38.47	−0.23 [−0.95, +0.47]	−0.14 [−2.04, +1.77]
R1: + uniform KL (ablation)	51.06	38.17	−0.32 [−1.02, +0.44]	−0.45 [−2.11, +1.44]
R3: + TGA + FGD	60.03	44.71	+8.66 [+7.66, +9.60]	+6.10 [+3.62, +8.96]
Published SAC 15× (20,000 steps)	54.95	39.26		

Table 3: SAC×ACON pilot on MRQA (Llama-3.2-1B, 15×, 5,000 steps, single seed, 6,786 evaluation examples). CIs are paired bootstraps over examples (1,000 resamples) and do not cover run-to-run variance.

FGD has no measurable effect: its intervals contain zero, and uniform KL is no better. A diagnostic run before training explains this: the zero-shot full-context teacher had a higher gold-answer NLL than the compressed student on 90.5% of 2,000 training examples, so there is little to distil. TGA + FGD improves both in-domain and out-of-domain F1, and after a quarter of the training budget already exceeds the published 20,000-step checkpoint. Because the question is seen during compression, the memory cannot be reused across questions, so this is a separate query-aware setting rather than a direct improvement on SAC. TGA without FGD has not yet been run.

4. Research Gaps

- Token savings do not become time savings.** ACON’s compressor generates autoregressively, and rewriting the history invalidates the agent’s KV cache, forcing the compressed prefix to be recomputed. ACON’s own measurement on AppWorld (Qwen3-14B agent and compressor, one A100) gives a median of 73.24s per task without compression, 87.68s with history compression and 101.92s with observation compression [1].
- The combined setting is schedule-confounded.** History compression fires when $|h_t| > T_{\text{hist}}$. Once observations are compressed first, $|h_t|$ grows more slowly, so history compression fires at different steps and on already-pruned text. ACON’s combined history+observation setting (44.6% accuracy for ACON_{UTCO}, versus 56.5% and 53.6% for history and observation compression alone [1]) therefore changes two things at once and is not a clean composition of two treatments.
- Compression quality is only measured indirectly.** ACON and SAC judge a compressor solely by downstream success or F1/EM. Neither measures whether the specific facts the agent will need survive compression, so a failure cannot be attributed to information loss rather than to the agent.

5. Problem Statement

Given a long-horizon agent with frozen weights, a history compressor and an observation compressor, determine how the two stages should be composed and scheduled so that the token savings of each are retained without lowering task success. The comparison between single-stage and composed settings must hold the compression schedule fixed, and compression quality must be measured directly, as the retention of task-critical state, rather than inferred only from task success.

Agent model. Following [1], a task is a POMDP $\mathcal{E} = \langle \mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{T}, R \rangle$. At step t an agent M with frozen parameters θ acts on the (possibly compressed) history h'_{t-1} and the latest observation o'_t :

$$a_t = M(h'_{t-1}, o'_t; \theta), \quad h_t = (o_0, a_0, o_1, a_1, \dots, o_t, a_t). \quad (1)$$

Compression operators. The observation compressor f_o and the history compressor f_h act as

$$o'_t = \begin{cases} f_o(o_t, h'_{t-1}) & |o_t| > T_{\text{obs}} \\ o_t & \text{otherwise} \end{cases} \quad h'_t = \begin{cases} f_h(x_t) & t \in \mathcal{K} \\ h'_{t-1} \oplus (o'_t, a_t) & \text{otherwise,} \end{cases} \quad (2)$$

where $\mathcal{K} \subseteq \{1, \dots, T\}$ is the set of trigger steps, $\oplus$ is concatenation and x_t is the input given to f_h . With $s < t$ the previous trigger step, the two possible composition orders are

$$x_t^{\text{seq}} = h'_s \oplus (o'_{s+1}, a_{s+1}, \dots, o'_t, a_t), \quad x_t^{\text{dec}} = h'_s \oplus (o_{s+1}, a_{s+1}, \dots, o_t, a_t). \quad (3)$$

ACON uses x^{seq} . A pipeline is the tuple $\pi = (f_o, f_h, x, \mathcal{K})$; we write $\pi_{\emptyset}$ for no compression, π_h and π_o for a single stage, and π_{ho} for both.

Schedule confound. In ACON the trigger set is determined by the pipeline itself:

$$\mathcal{K}(\pi) = \{t : |h'_{t-1} \oplus (o'_t, a_t)| > T_{\text{hist}}\}. \quad (4)$$

Because $|o'_t| \leq |o_t|$, in general $\mathcal{K}(\pi_{ho}) \neq \mathcal{K}(\pi_h)$. Let $\Delta(\pi) = \mathbb{E}[R(\pi)] - \mathbb{E}[R(\pi_{\emptyset})]$ and define the interaction

$$I = \Delta(\pi_{ho}) - \Delta(\pi_h) - \Delta(\pi_o). \quad (5)$$

For ACON_{UTCO} on AppWorld [1], $\Delta(\pi_h) = +0.5$, $\Delta(\pi_o) = -2.4$ and $\Delta(\pi_{ho}) = -11.4$ points, so $I = -9.5$. Under Eq. (4), I mixes the effect of composing f_o with f_h and the effect of the shifted triggers, and the two cannot be separated.

Cost and latency. Token cost, peak context and wall-clock latency are

$$C(\pi) = \sum_{t=1}^T (|h'_{t-1}| + |o'_t|), \quad P(\pi) = \max_t (|h'_{t-1}| + |o'_t|), \quad (6)$$

$$L(\pi) = \sum_{t=1}^T \ell_M(h'_{t-1}, o'_t) + \sum_{t \in \mathcal{K}} \ell_{f_h}(x_t) + \sum_{t: |o_t| > T_{\text{obs}}} \ell_{f_o}(o_t), \quad (7)$$

where ℓ denotes per-call latency. ℓ_M includes re-encoding the prefix after each trigger, since rewriting h'_t invalidates the KV cache. The last two sums are overhead that $C(\pi)$ does not capture.

Critical-fact retention. Let $V(\cdot)$ extract state variables (access tokens, entity IDs, file paths, numeric values) from text, and let $a_{1:T}^{\emptyset}$ be the actions of the uncompressed reference run. The facts at stake in trigger t and the retention rate are

$$F_t = V(x_t) \cap \bigcup_{t'} V(a_{t'}^{\emptyset}), \quad \text{CFR}(\pi) = \frac{\sum_{t \in \mathcal{K}} \sum_{v \in F_t} \mathbf{1}[v \sqsubseteq h'_t]}{\sum_{t \in \mathcal{K}} |F_t|}, \quad (8)$$

where $v \sqsubseteq h'_t$ denotes an exact (verbatim) occurrence of v in h'_t .

Formal problem. For $\lambda, \mu \geq 0$ and tolerance $\varepsilon \geq 0$, find

$$\pi^* = \arg \max_{x \in \{x^{\text{seq}}, x^{\text{dec}}\}, \mathcal{K}} \mathbb{E}[R(\pi)] - \lambda \mathbb{E}[C(\pi)] - \mu \mathbb{E}[L(\pi)] \quad (9)$$

s.t. $\mathbb{E}[R(\pi)] \geq \mathbb{E}[R(\pi_{\emptyset})] - \varepsilon$, $C(\pi_{ho}) \leq \min\{C(\pi_h), C(\pi_o)\}$, $\mathcal{K}(\pi_{ho}) = \mathcal{K}(\pi_h)$ when comparing π_{ho} with π_h , and report $\text{CFR}(\pi)$ alongside $\mathbb{E}[R(\pi)]$ to test whether loss of reward is explained by loss of critical facts.

6. Proposed Solution

We propose three changes to ACON’s pipeline, each addressing one term of the problem in Section 5 (Figure 1).

1. Decoupled composition order. The observation compressor still produces a short o'_t for the agent’s immediate step, but the raw o_t is retained in a side buffer and f_h receives x_t^{dec} instead of x_t^{seq} (Eq. (3)). If f_o only deletes content, then $V(o'_t) \subseteq V(o_t)$, so F_t under x^{dec} contains F_t under x^{seq} : facts dropped by the observation stage can still enter the history summary.

2. Controlled compression schedule. We record $\mathcal{K}^h = \mathcal{K}(\pi_h)$ from the history-only run and replay it, setting $\mathcal{K}(\pi_{ho}) := \mathcal{K}^h$ instead of recomputing Eq. (4) on pruned text. With the schedule fixed, π_h and π_{ho} differ only in f_o , so the interaction I in Eq. (5) reflects composition alone.

3. Critical-Fact Retention (CFR). We implement $V(\cdot)$ with pattern extractors for AppWorld’s tokens, IDs, paths and numbers, and compute Eq. (8) offline for every compressor call, with no additional agent rollouts. Across tasks we then measure the rank correlation $\rho(\text{CFR}, R)$ to test whether CFR predicts task success, and compare CFR under x^{seq} and x^{dec} .

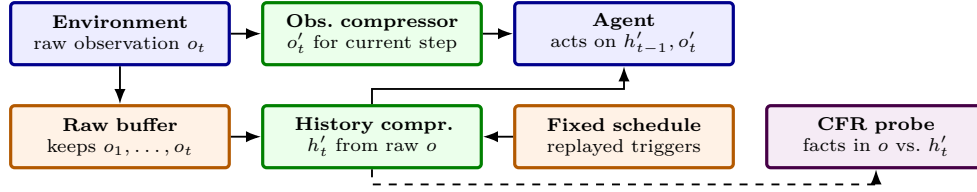


Figure 1: Proposed pipeline. The history compressor reads raw observations from a side buffer (orange), fires on a schedule fixed across conditions, and every call is scored by the CFR probe (violet).

7. Future Work

- **Complete the ACON runs.** Finish the 38-task history-compression run, and run observation compression, combined history+observation compression, the paper thresholds ($T_{\text{hist}} = 4096$, $T_{\text{obs}} = 1024$) and the llama3.1:8b compressor on the same split.
- **Implement the proposed pipeline.** Add the raw-observation buffer (x^{dec}), schedule replay ($\mathcal{K}(\pi_{ho}) := \mathcal{K}^h$) and the CFR extractor, then estimate I under both composition orders with the schedule fixed.
- **Measure latency directly.** Log ℓ_M , ℓ_{f_h} and ℓ_{f_o} per call to decompose $L(\pi)$ in Eq. (7), including prefix re-encoding after each history compression.
- **Strengthen the evidence.** Use at least three seeds per setting, raise the served context window above 4096 tokens so baseline prompts are not truncated, and, for SAC×ACON, run TGA alone together with the 5× and 51× ratios.

References

- [1] M. Kang, W.-N. Chen, D. Han, H. A. Inan, L. Wutschitz, Y. Chen, R. Sim, and S. Rajmohan, “ACON: Optimizing Context Compression for Long-horizon LLM Agents,” in *Proc. 43rd Int. Conf. Machine Learning (ICML)*, PMLR 306, 2026.
- [2] X. Liu, R. Zhao, P. Huang, X. Liu, J. Xiao, C. Xiao, T. Xiao, S. Gao, Z. Yu, and J. Zhu, “Autoencoding-Free Context Compression for LLMs via Contextual Semantic Anchors,” in *Int. Conf. Learning Representations (ICLR)*, 2026.
- [3] Y. Wang, Y. Shi, M. Yang, R. Zhang, S. He, H. Lian, Y. Chen, S. Ye, K. Cai, and X. Gu, “SWE-Pruner: Self-Adaptive Context Pruning for Coding Agents,” arXiv:2601.16746, 2026.
- [4] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, “LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models,” in *Proc. EMNLP*, pp. 13358–13376, 2023.
- [5] Z. Zhou, A. Qu, Z. Wu, S. Kim, A. Prakash, D. Rus, J. Zhao, B. K. H. Low, and P. P. Liang, “MEM1: Learning to Synergize Memory and Reasoning for Efficient Long-Horizon Agents,” arXiv:2506.15841, 2025.
- [6] H. Yu, T. Chen, J. Feng, J. Chen, W. Dai, Q. Yu, Y.-Q. Zhang, W.-Y. Ma, J. Liu, M. Wang, and H. Zhou, “MemAgent: Reshaping Long-Context LLM with Multi-Conv RL-based Memory Agent,” in *Int. Conf. Learning Representations (ICLR)*, 2026.
- [7] H. Trivedi, T. Khot, M. Hartmann, R. Manku, V. Dong, E. Li, S. Gupta, A. Sabharwal, and N. Balasubramanian, “AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents,” in *Proc. ACL*, pp. 16022–16076, 2024.